

PROCEDURAL DUNGEON GENERATION IN ROGUELIKE GAMES

Github Repo: <https://github.com/Sumaylen/Final-PCG-Project>

Note: My project was made using Unity's built in version control; therefore, there is only one commit on GitHub as it was only used for submission.

Total Word Count: 7218/10500

Template: Template 6.2 Project Idea 2: Procedural Dungeon Generation in Roguelike Games

INTRODUCTION

Word Count: 244/1000

My project is based on Template 6.2 Project Idea 2: Procedural Dungeon Generation in Roguelike Games.

Procedural content generation (PCG) refers to the use of algorithms to automatically generate content, rather than relying on the traditional method of manually designing it [3]. It can increase player engagement and reduce the workload for game designers while allowing for gameplay that feels unique on each playthrough. PCG has been used in the games industry for a long time, especially in the roguelike genre. Beneath Apple Manor [4], a dungeon exploration roguelike game written by Don Worth for the Apple II in 1978, was one of the first roguelike games to make use of PCG. It did so by generating random dungeon floors and monster placements, allowing for each play session to be a fresh one. The use of PCG was a relatively new technique at the time, and this laid the foundation for the replayability that roguelike games are known for today.

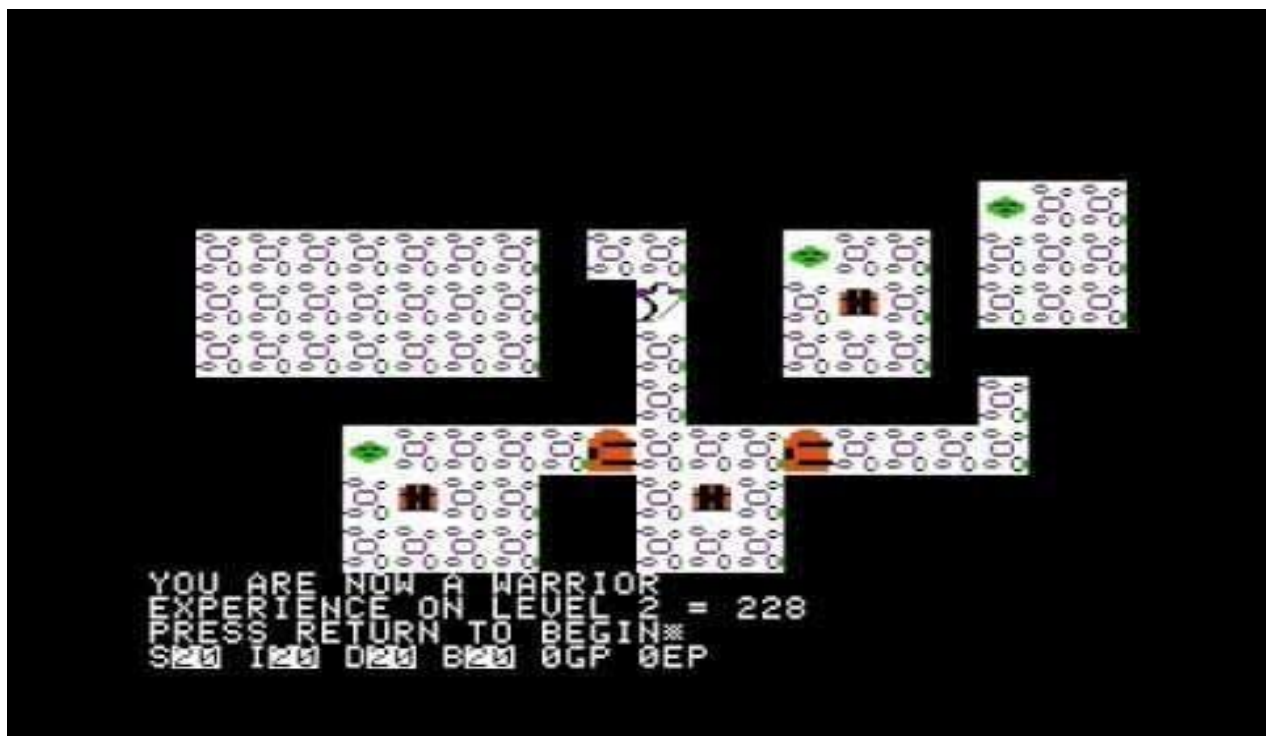


Figure 1 Beneath Apple Manor on the Apple II [2]

However, PCG often involves a balance between randomness and structure. A purely random dungeon often lacks the playability required for a satisfying user experience. When creating a procedurally generated dungeon, game designers often face the challenge of making the layout feel natural; therefore, algorithmic techniques are needed to address this issue. The goal of this project is to explore the techniques and implementation of two PCG algorithms: Binary Space Partitioning and Cellular Automata and compare their effectiveness in creating a replayable roguelike gameplay experience.

Literature review

Word Count: 580/2500

Procedural content generation (PCG) has become increasingly widespread in game development because it enables highly replayable content to be generated efficiently. Creating large amounts of content manually can be costly and difficult to scale; therefore, rather than just relying on human designers, PCG can be used to generate content algorithmically. This means game designers can generate game environments more efficiently without losing control over the final product by using parameter settings and constraints. This makes PCG a valuable method for creating new experiences without requiring every aspect of the game to be handcrafted.[1]

Procedural content generation (PCG) provides two advantages to game design. First, PCG enables the generation of nearly limitless content and variation for each play session. Second, PCG allows for the creation of entirely unique player experiences. When the gameplay encourages interaction with the game systems rather than relying only on hand-scripted events, players can create personal narratives unique to them. This approach also supports unpredictable gameplay situations, allowing for a wider

range of playstyles that may not have been planned by developers. Game content with high degrees of randomness and variation often facilitates these unique experiences, as individual playthroughs can be shaped by unexpected interactions between the player and the game. [2]

One way to implement PCG in a video game is through algorithmically generating maps, allowing for gameplay experiences that vary from run to run. Binary Space Partitioning and Cellular Automata are examples of such algorithms

Binary Space Partitioning:

Binary Space Partitioning is a method of dividing a space into smaller sections by recursively splitting it into two parts [3]. It can be used to generate map layouts by creating rooms and corridors within a defined area.

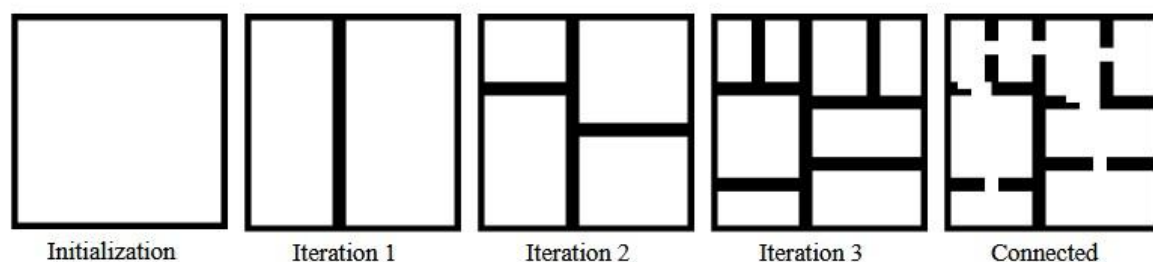


Figure 2 Binary Space Partitioning (BSP) Iterations [1]

BSP was originally popularized in the gaming industry by John Carmack for its use in solving the Visible Surface Determination (VSD) problem in Doom (1993) [5]. VSD is the process of deciding for every pixel on the screen which part of the 3D world geometry is visible to the camera and which parts are hidden by closer objects. As documented in Michael Abrash's Graphics Programming Black Book [6], BSP trees were used to sort 3D polygons from front to back, thus optimizing rendering on the limited hardware of the time. While its original use was for determining visibility (rendering), the same recursive logic can be used for generating map layouts.



Figure 3 Doom (1993) on MS-DOS

One of the main strengths of BSP is that it results in a consistent output. The algorithm ensures that the map can be fully navigated by preventing rooms from overlapping. However, this approach also constitutes a considerable limitation as BSP can result in a very structured and monotonous layout that lacks the organic feel of a hand-crafted level.

Cellular Automata:

Cellular automata is a simulation technique that operates on a grid of cells, where each cell's state is updated repeatedly based on a set of rules and the states of its neighbouring cells [3]. With each pass, the new state replaces the previous one, allowing for patterns to emerge over time. CA offers an effective procedural method for generating unique and natural-looking cavelike layouts.

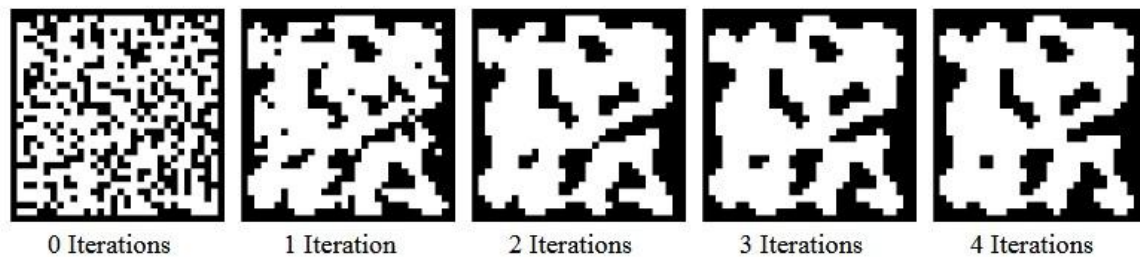


Figure 4 Celulalr Automata (CA) Iterations [1]

Cellular automata were originally proposed by John von Neumann as formal models of selfreplicating organisms and were subsequently brought to wider attention in Conway's Game of Life when it was introduced in 1970 [7]. Conway's Game of Life is a computational simulation played on a grid, where each square can either be occupied or empty and evolves over time. Conway initially presented it as a board-based game before it was implemented as a computer program. The evolution of the grid is governed by three basic conditions: a living cell continues to the next generation only if it has two or three adjacent living cells; it dies if there are too few or too many neighbours and an empty cell becomes occupied if it has exactly three neighbours.

Some of the benefits of using cellular automata as a PCG technique is its simplicity and ability to produce natural-looking layouts on smaller grids. The algorithm generates unique structures that feature connected chambers and corridors. However, CA relies heavily on probabilistic rules, which can lead to a lack of control and unpredictable results, such as isolated caves. Furthermore, its effectiveness tends to diminish on larger maps, where the results often become less aesthetically pleasing. A possible solution might be to break down larger grids into smaller ones before combining them or to make use of multiple PCG techniques together.

Procedural dungeon generation does not have a single universally superior method. Instead, different techniques offer their advantages depending on the game's design goals. There have been a wide range of approaches to PCG in video games, each with its own strengths and limitations [8]. This supports my project goal, since both BSP and CA can generate replayable levels but differ enough to offer contrasting gameplay

experiences. Therefore, by combining both BSP and CA into a single game, I can better compare their respective strengths.

3. Project Design

Word Count: 1536/2000

Project Name: Shifting Depths

1. Overview of the Project

This project aims to develop a playable roguelike prototype that implements two procedural map generation techniques: Binary Space Partitioning (BSP) and Cellular Automata (CA), along with gameplay systems such as combat, enemy encounters, player progression, and dynamic difficulty scaling. The prototype will generate dynamic 3D map layouts while providing players with a balanced, replayable gameplay experience. As a result, I will be able to compare these map-generation techniques and examine their effects on players.

2. Domain and Target Users

Domain

The domain for this project is primarily procedural content generation in game development, and its application within the roguelike genre. Games in this domain depend on replayability and run-to-run variation between playthroughs but must remain coherent enough to be navigated and played without any issues. Therefore, a good PCG game needs to generate content algorithmically while ensuring that the resulting levels remain fair, balanced, and playable.

Target Users

The main users are roguelike players, especially those who enjoy a replayable dungeon-crawler experience with hack-and-slash combat. These players usually expect an engaging experience with minimal repetition, where each subsequent run is noticeably distinct from the previous one. A secondary audience is researchers and game developers interested in procedural content generation techniques and their effects on gameplay experience. For these users, the project provides full control over map generation through the Unity Inspector, enabling different settings to be tested and their effects observed. Another useful feature for game designers is the log metrics, which provide information about map generation, such as the number of rooms or the % of the map that can be navigated.

The design must balance replayability with playability by ensuring that generated levels are traversable and fair, rather than entirely random. This is achieved by implementing

progression systems that provide players with options beyond simple layout variation, thus allowing for different playstyles for different situations. Additionally, the map-generation techniques must produce distinct visuals and player experiences to allow for meaningful comparison. This guarantees an engaging experience for players and supports comparison and evaluation of PCG techniques for game designers.

3. Project Structure and Justification

3.1 Gameplay Loop

The gameplay loop revolves around replayable dungeon-crawling runs. At the start of a run, the player is dropped into a random room of a procedurally generated dungeon. They are then tasked with activating a disabled portal. To do this, they must explore the maze-like layout to find and collect keys to power the portal. While doing this, they will have to partake in hack-and-slash combat with enemies blocking their path. Further pressure is added by a ticking timer that makes enemies stronger the longer the player takes; if the player takes too long, even stronger enemy types start appearing. All is not lost for the player, though, since as they defeat enemies, they grow stronger and can choose from various stat level-ups to boost their combat capabilities further. Killing enemies also sometimes drops temporary power-ups, further assisting the player. If they collect all the keys, they can then progress to the second stage through the now-activated portal. This second stage is significantly different from the first in both layout and theme, with one simple goal: “survive”. If the player is left standing at the end of the timer, they win. This loop of explore → combat → reward forms the basis of my game and is typical of the roguelike genre.

3.2 Map Generation

Procedural map generation is a core feature of my project, as it is required to generate dynamic layouts that ensure each run feels different from the last. To accomplish this, I implemented two distinct procedural generation techniques.

Binary Space Partitioning (BSP) was chosen as the technique for my first stage because it produces a more structured layout with well-defined rooms of various sizes and corridors connecting them. This creates an easily navigable map with strong connectivity, ensuring all areas are accessible to the player. Exploration is a key aspect of this map, as players must explore each room to find the key. Therefore, BSP provides a typical dungeon-crawling experience.

For the second stage, I chose to use Cellular Automata. It creates a more organic cave-like layout with little structure. The player must adapt to the chaotic changes in the environment; sometimes they are in a large, open room, other times they are navigating narrow, winding corridors.

These two techniques offer contrasting gameplay experiences and separating them into

stages makes it easier to compare their impact on players' experiences based on feedback and surveys.

3.3 Gameplay

The moment-to-moment gameplay determines how players interact with the generated environments. Combat and player progression are the main features of the gameplay. During a run, the player engages in hack-and-slash combat, utilising timed attacks and movement to defeat enemies that block their path. The combat system requires the player to manage positioning and movement to avoid enemy attacks, while player progression offers different playstyles, further enhancing replayability.

3.3.1 Player Progression, Stats, and Items

For every enemy they kill, the player earns experience. Once they have enough experience, they level up, allowing them to choose one of four stats to upgrade, which will enhance their combat capabilities. These stat upgrades include increasing max health, attack damage, attack speed, and movement speed. Each stat upgrade comes with its own advantage, allowing players to build their character how they want. Players can choose to fully invest in one stat or spread their upgrades amongst all stats, allowing for a more balanced playstyle. The amount of experience required to level up increases with every level, further emphasising the importance of the player's choices. Randomness and luck are also introduced during level-ups to add further variation to each run. This is achieved through 3 rarity tiers, randomly selected for each upgrade: common (white), rare (blue), and epic (purple). The higher the rarity, the stronger the stat upgrade.

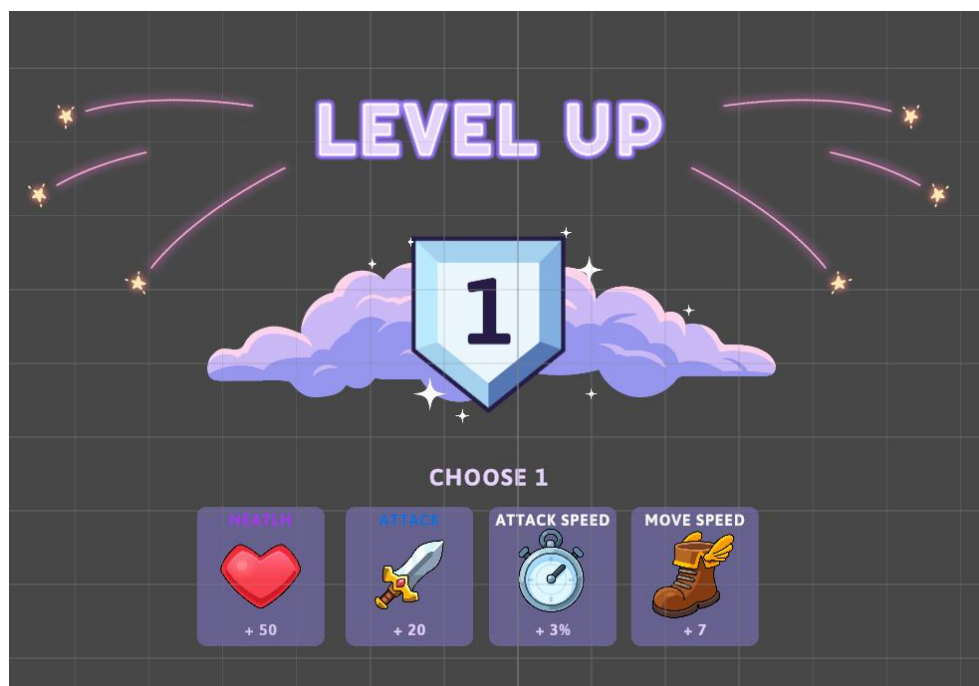


Figure 1 Level-up banner showcasing upgrade options and rarity tiers.

Killing enemies also has a low chance to drop items that further assist the player. These items include a heart, which heals lost health, a skull, which provides a temporary increase in attack damage, and a lightning bolt, which provides a burst of movement speed. These item drops are key to assisting players through the run, and the random chance to drop makes it feel more rewarding when it happens.



Figure 2 3D models of the rare item drop from killing enemies

3.3.2 Enemies and Difficulty Scaling

Enemies are the main opposition players face and the source of experience points. In stage 1, enemies will appear in all rooms besides the start room and will only chase the player if they get nearby. While in stage 2, they will continue to appear near the player as a relentless wave. Enemies are split into 2 types: the regular enemies, which are most common, and elite enemies, which are rarer but much stronger.

The difficulty scaling primarily revolves around the timer: as it ticks down, the game becomes harder; enemies have more health, do more damage, and there is an increased chance of elites appearing.

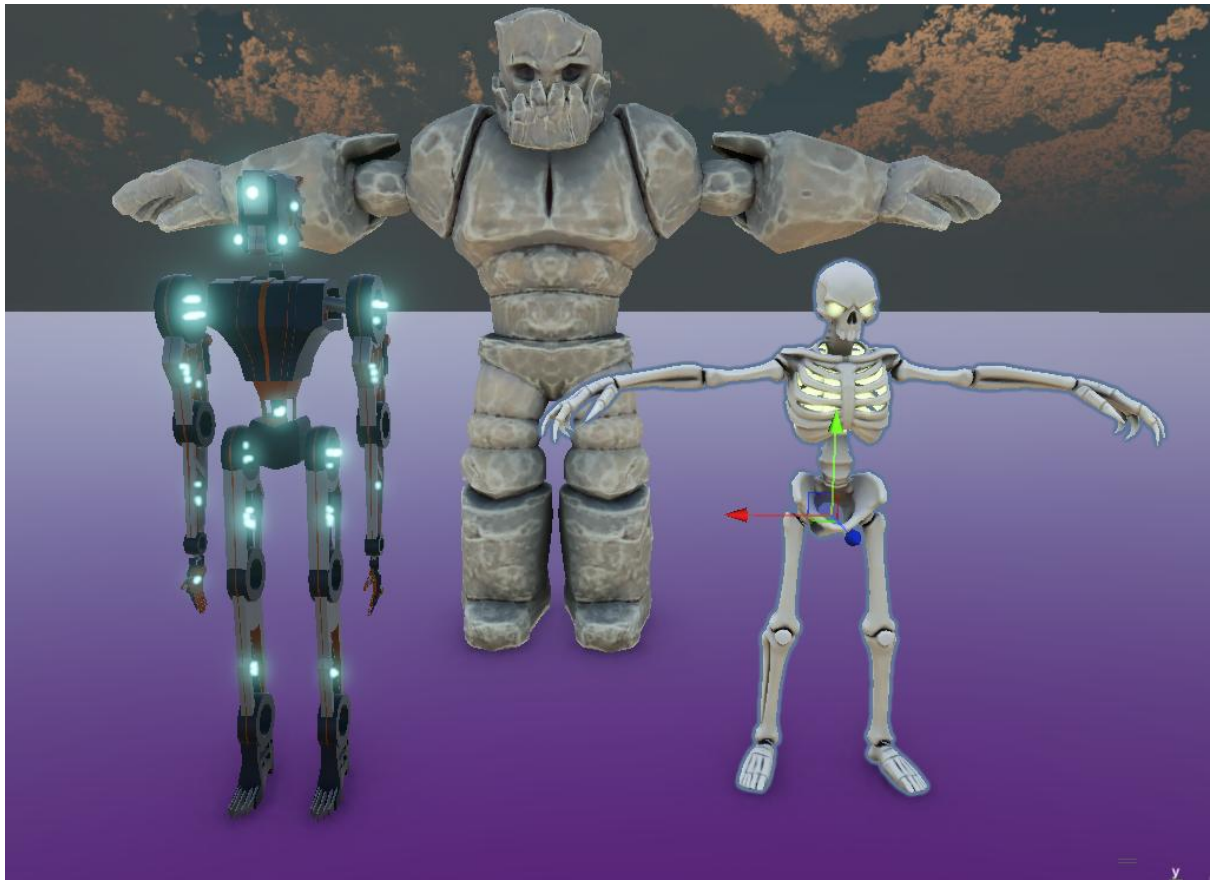


Figure 3 3D models of the 2 regular enemies and 1 elite

3.4 Theme, Setting, and Presentation

My game is set in a fictional universe that blends fantasy and sci-fi. The player controls a knight in armour, wielding their magical, flaming sword, and is tasked with exploring a shifting dungeon where each floor exists in its own dimension.



Figure 4 3D model of the playable character in armour with their magical flaming sword

Stage 1 follows the typical high-fantasy dungeon, with stone walls and dim light providing an eerie setting one might expect in a crypt. The enemies on this stage further reflect this setting, with regular enemies being skeletons and elite enemies being magical stone giants.

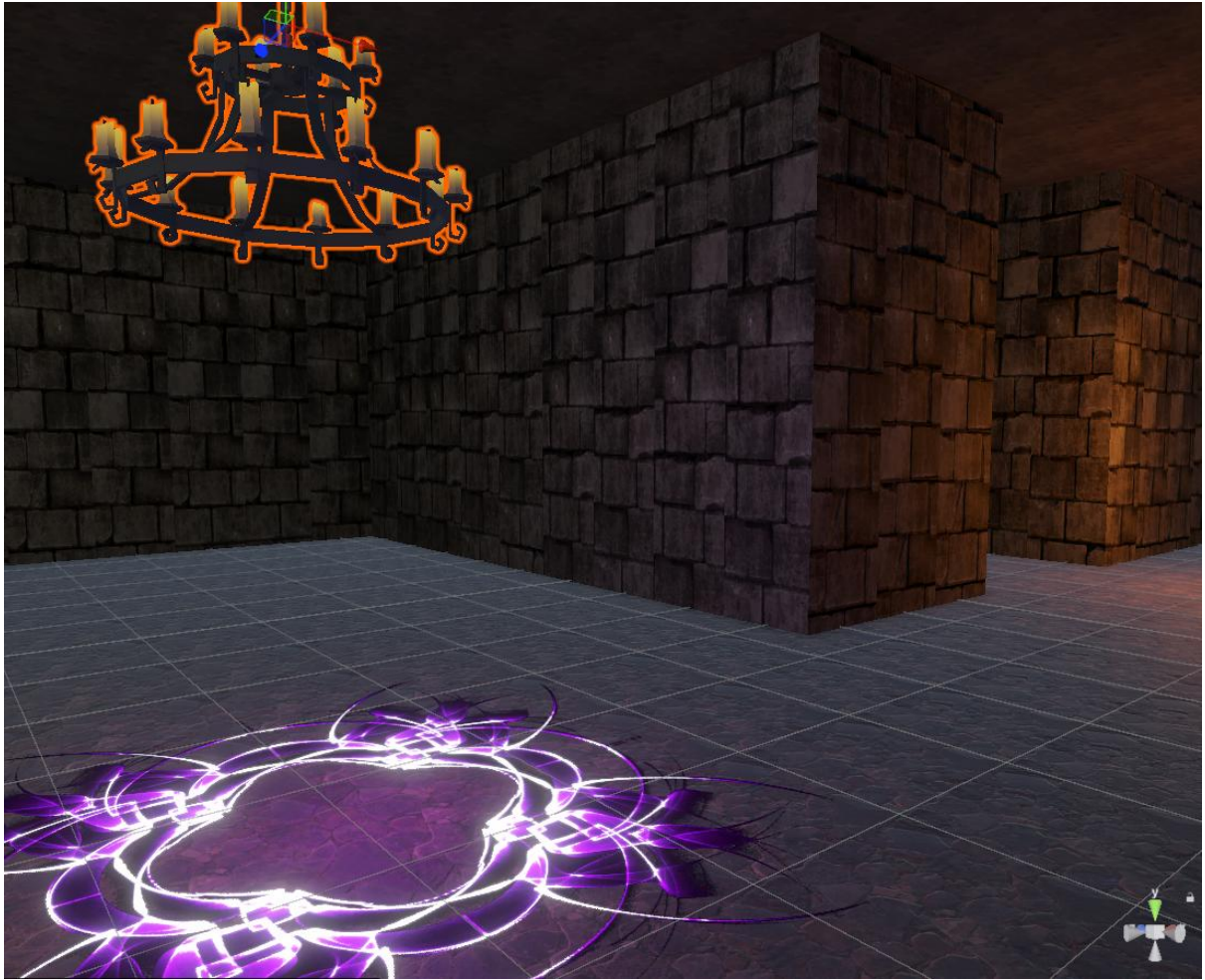


Figure 5 Crypt like map theme for stage 1

Stage 2 contrasts this by shifting to a more sci-fi setting: the map is more open, with no ceiling, revealing the celestial sky; the walls are glowing and futuristic, while the enemies are high-tech robots.

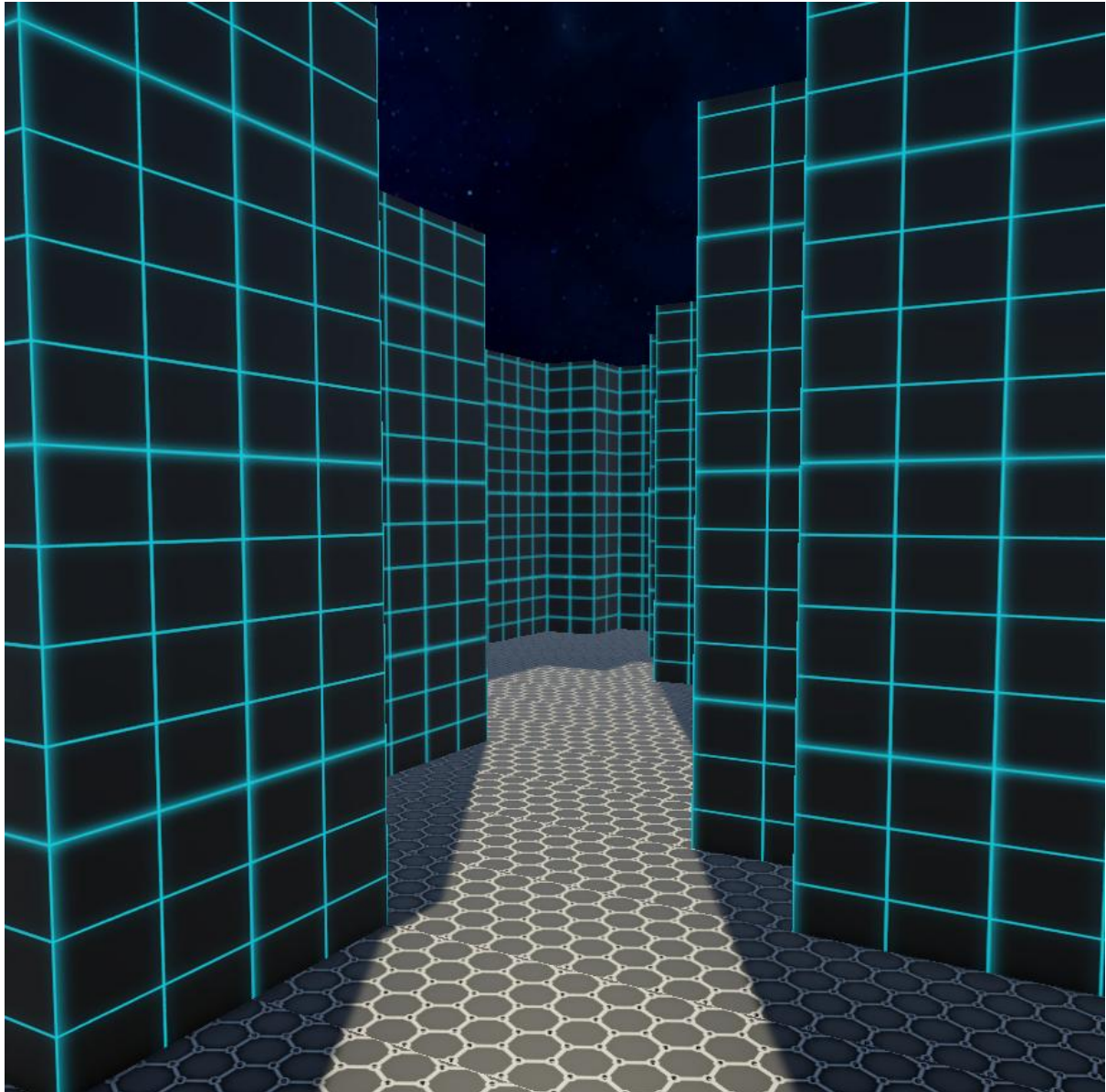


Figure 6 Futuristic map theme for the stage

The players attacks also include an arc visual effect that happens when they attack further providing visual feedback.



Figure 7 Animation and arc feedback for a sword swing

3.5 User Interface

The first UI element presented to the player is the main menu. The background shows the entrances to the explorable dungeon where the game takes place, and the title card showcases the game's name. Players can either initiate a run from the start by clicking play or select a specific stage to start the game. This is particularly important, as it allows players to choose which procedural map generation to try immediately.



Figure 8 Main Menu of the game

Once in-game, players are presented with the HUD, which includes the health bar, current stat values, countdown timer, experience bar and a mini-map. The mini map is a noteworthy addition because it shows the player's and portal's positions in real time, ensuring they never get lost while exploring. The stat display is also essential for indicating the current strength of the player's attributes.

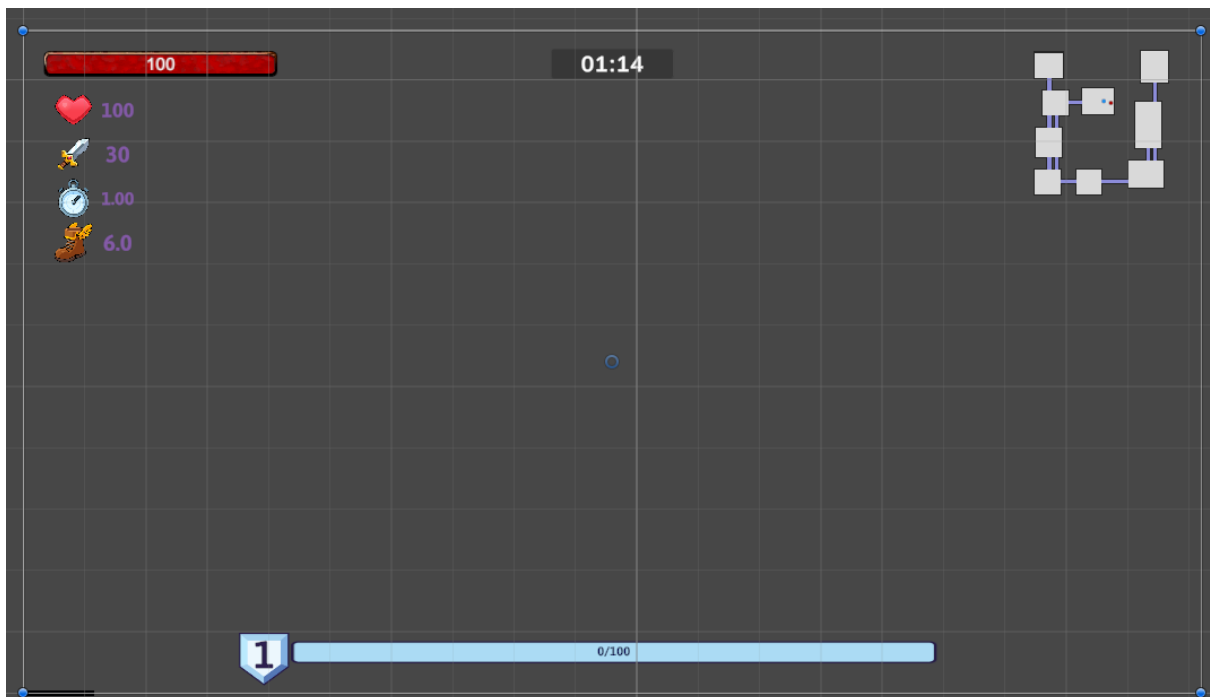


Figure 9 Heads-up display (HUD) of the player

The pause menu is the final major UI element that allows players to pause the game and return to the main menu. Pausing the game also brings up the log metrics, which show various technical stats about the game, especially useful to players curious about map generation or to view gameplay stats such as total kills.



Figure 10 Pause menu and Log Metrics

3.6 Technical Design Decisions

The prototype is designed in Unity 6 because it provides flexibility for implementing procedural generation, 3D rendering, and prototyping features, which are essential to this project. Unity includes a wide range of plugins, asset store resources, and documentation, which further support my project needs. My prior experience with Unity reduces the learning curve, while its widespread use in indie games, especially in the roguelike genre, ensures compatibility with industry standards, enabling future scalability. Blender is also used as a supplementary tool for the editing of certain 3D models.

4.Implementation

Word Count: 2468/2500

Introduction

This chapter explains how the prototype was implemented using Unity 6. It focuses on the main systems used by the project and together they form a playable roguelike prototype

4.1 Binary Space Partitioning (BSP)

BSP works by recursively dividing a larger space into smaller subspaces. In my project, these spaces are used to place rooms, which are then connected by corridors.

BSPDungeonGenerator handles the main map generation process. The GenerateDungeon() method first creates a 2D grid to store the dungeon map and creates a root BSPNode covering the whole map. A BSP node represents a section of a binary tree, where each node can have up to two child nodes and stores a rectangular section of the map. BSPDungeonGenerator then calls SplitNode(), which recursively divides the given space into smaller partitions. This recursive splitting stops when the partition is small enough or when the tree's maximum depth is reached. This maximum depth is the number of subdivision levels the dungeon can have; the higher the depth, the more times the dungeon is divided. The direction the dungeon is split is random, either horizontally or vertically but the method also considers the shape of the partition. Wide areas are more likely to be split vertically, while tall areas are more likely to be split horizontally. This helps keep the partitions' shapes more balanced, thus avoiding rooms that are too long or too wide.

After the recursive splitting ends and spaces are divided, CreateRooms() is called. This function traverses the BSP tree and places rooms only at leaf nodes (i.e., nodes with no child nodes), so rooms are placed only in the final partitions. The size and position of each room are random, but they remain within the bounds of the partition and leave a padding around the edges so there is space for walls and corridors. Once a room is created, its tiles are marked as floor cells in the grid. A random value is also used to determine whether a partition becomes a room, helping prevent the creation of too many rooms and adding some variance.

ConnectRooms() is then called to connect the newly created rooms. The function finds a room for each child node of a parent and connects them with a corridor. The corridor is created by connecting the centres of the rooms and assigning the grid cells between

them as corridors.

The result is a 2D grid of cells, each marked as a room, a corridor, or empty space. DungeonBuilder then turns it into a playable 3D level. It reads the grid type and places different prefabs in their correct positions. It also checks the tiles around the rooms and corridors to place walls on the empty cells surrounding them. The script also adds lights to the centre of each room and builds a NavMesh so AI enemies can traverse the dungeon.

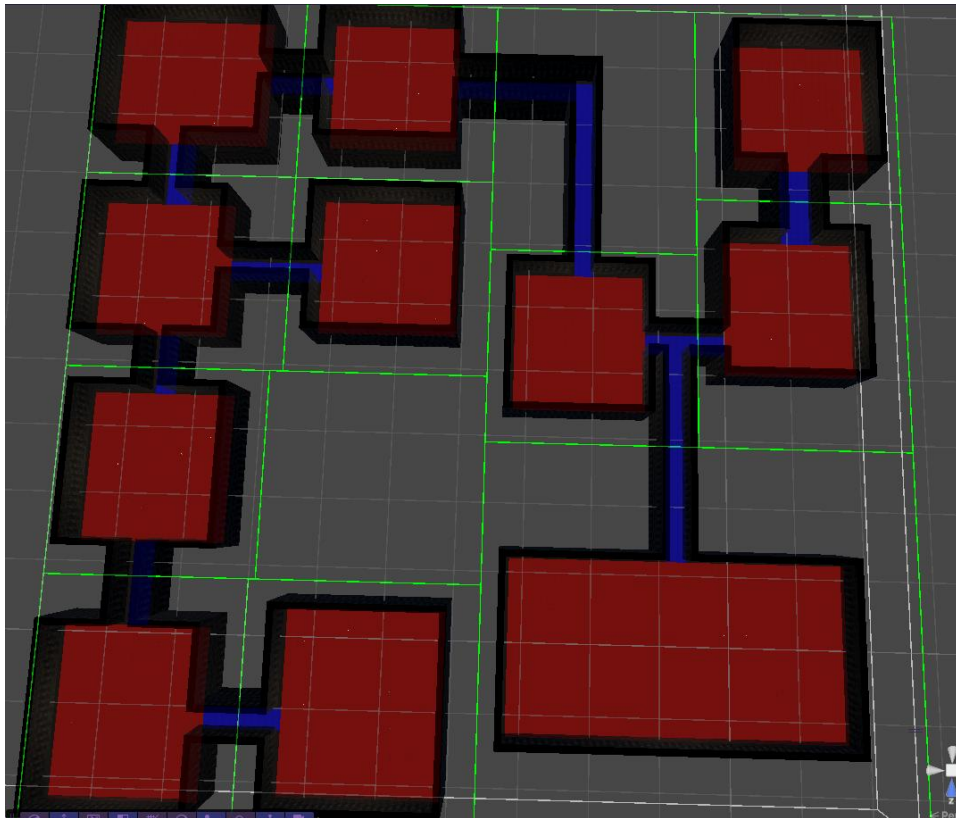


Figure 11 BSP generated dungeon clearly highlighting the partitions, rooms and corridors

4.2 Cellular Automata (CA)

CA works by repeatedly updating a grid of cells based on the state of its neighbouring cells. The result in my project is a more organic cave-like layout.

CADungeonGenerator is responsible for generating the dungeon layout and converting it into a 3D environment. It begins by creating a 2D array that stores the dungeon as wall and floor cells. The outer edges of the map are always set as walls, while every internal cell is initially randomly assigned as either a floor or a wall, determined by a fillProb variable. This forms the initial state of CA, which is rough and uneven and is ready for smoothing.

Smoothing is an iterative process, meaning it is run multiple times on the grid state. The

number of times the smoothing function runs is determined by the `smoothingSteps` value. During an iteration, every cell in the grid is checked and compared to its surrounding cells. These neighbouring cells are checked to see if they are walls. Cells that are out of bounds of the dungeon are automatically considered as walls. If the number of walls surrounding the initial cell exceeds the threshold set by the `wallRequirement` variable, that cell becomes a wall as well. If it is less than that, the initial cell is set as a floor; otherwise, if the number of walls is equal, nothing happens. This process is then repeated a specified number of times, resulting in greater noise reduction and larger connected regions.

After the cells have been defined and the smoothing steps are run, we are left with a 2D grid of floors and walls. `Build3DMap()` is called to assign floor or wall prefabs depending on the cell type, thereby converting the 2D grid into a 3D map. It also sets the floor cells as a NavMesh surface, allowing for AI pathfinding.



Figure 12 Top view of a CA map clearly showing the wall and floor cells

4.3 Progression, Stats and Item Drops

`PlayerProgression` manages the player level and experience. When an enemy is defeated, the player is awarded experience based on the type of enemy killed. `AddXP()`

checks if the player meets the required XP threshold to level up. When the player's experience reaches the required threshold, their level increases, with any excess experience carried over to the next level. The XP threshold scales exponentially, using the following formula:

$$XP_{required} = \lceil nextXP \cdot (xpMultiplier)^{(lvl-1)} \rceil$$

meaning that the higher the player's level, the more experience is required to reach the next level.

PlayerStats stores the main gameplay attributes of a player, such as melee damage, movement speed, maximum health and AttackSpeed bonus. It also allows these stats to increase during level-ups. All stats are increased by a flat amount, except for AttackSpeed, which had to be further balanced with diminishing returns. This is done through the formula:

$$increase = \frac{baseAttackSpeedGain}{1 + currentAttackSpeedBonus}$$

meaning the greater the bonus, the less impactful the upgrade is. It also handles temporary stat buffs from pickups.

The chance for an item to drop on enemy death is determined in the ItemDrop script, where a probability check is run. If successful, a random prefab from a predefined array is dropped at the position of the enemy's death. Each droppable item can be picked up by the player when the player enters close proximity to the item. These buffs are applied by increasing the respective stat, and a coroutine is started that waits for a specified duration before reverting that stat to what it was before the buff was picked up. This allows for temporary buffs to be applied without permanently changing the base stats.

When the player levels up, the UpgradeSystem script calculates four available stat upgrades. These upgrades are determined by a base upgrade amount, with some random variance applied. These are then further potentially increased based on their rarity. Rarity is randomly selected based on probability; the higher the rarity, the lower the chance for it to be rolled. It determines the multiplier applied to the stat upgrade, except for AttackSpeed, where it controls the number of times the AttackSpeed upgrade is applied. Each upgrade option can be chosen only once, and the rest are discarded.



Figure 13 Randomly generated stat upgrades and rarities

Since my game has two stages, persistence of player stats, experience, and level data between scene transitions and on level retries was required. This is implemented in the PlayerData script: the SavePlayer() function stores the overall player state whenever player values are permanently changed, and the SaveStage() function serves as a checkpoint, storing a copy of the saved state on stage change.

4.4 Enemy Spawning and Artificial Intelligence

Enemy spawning depends on the dungeon generation method. For the BSP stage, spawning is handled by the EnemySpawnerBSP script. In this script, a list of rooms is collected from the BSP tree, and each is assigned a room state. The player's spawn room is excluded as a valid spawn point to prevent enemies from appearing in the room the player starts in. Each room keeps track of the number of enemies currently alive and a timer for the next enemy respawn. When the map is first generated, each room is filled with the target number of enemies. There is a continuous check whether a room has fallen below the minimum number of enemies, and if so, attempts to spawn a new enemy inside that room, depending on the respawn delay timer.

A random point within the room is chosen, but a padding prevents enemies from spawning too close to the wall. A layer mask is also set and checked by the Physics.OverlapSphere() so that enemies do not spawn on the blocked layer, such as other enemies and the player. If the spawn position is valid, the enemy prefab is instantiated in that position. The script also randomly determines if the instantiated enemy is an elite based on the eliteSpawnChance variable.

For the CA stage, EnemySpawnerCA tries to maintain a minimum number of enemies around the player. If the current number of enemies drops below the minimum and the respawnDelay timer has ended, it attempts to spawn an enemy. This spawn location is a random position within a specified spawn radius. This location is checked to see if it's on a valid NavMesh using NavMesh.SamplePosition(), which would mean it's a floor cell. Next NavMesh.CalculatePath() is called to check whether there is an uninterrupted path from that location to the player. This ensures that the enemy isn't spawned in a blocked-off part of the map. If all these checks pass, an enemy is instantiated in that position.

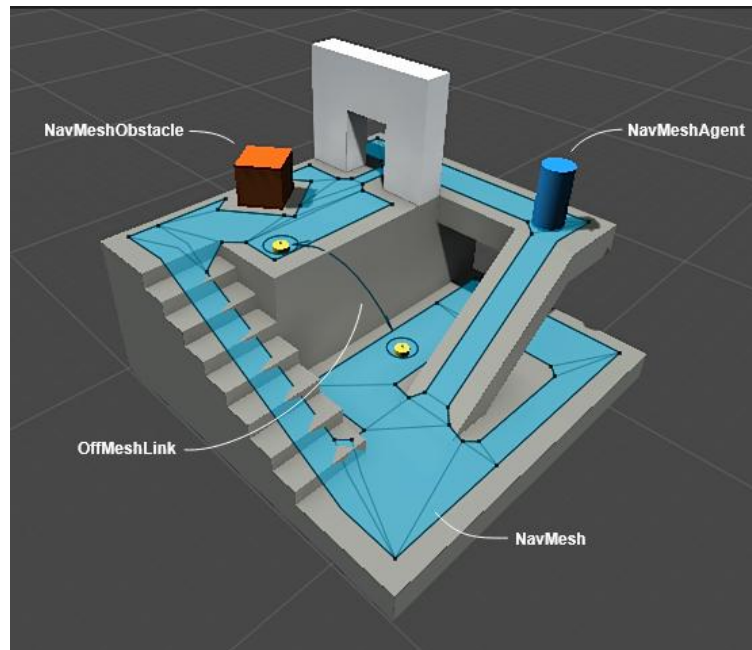


Figure 14 Diagram showing a NavMesh system implemented in Unity

In the EnemyAI script, enemy movement and player detection are handled. Enemies use a NavMeshAgent for navigation, allowing them to move on any NavMeshSurface created during map generation. Enemies have two main states: wandering and aggressive. In the BSP stage, enemies are assigned to the room they spawn in. Valid random points are selected in this room, and enemies will wander between these points based on the nextWanderPoint variable.

Aggro state is determined by distance to the player. If the player enters the enemy's aggro radius, the enemy switches to an aggressive state, and the NavMeshAgent continuously updates its destination to the player's location. A second radius variable, chaseRadius, determines how far the player must be from the enemy for the enemy to exit this aggressive chase state and return to their room, unless the keepChasingOnceAggro option is enabled.

The EnemyCombatMelee scripts check if the enemy is within melee radius using a distance calculation. When players are in range, and the attack isn't on cooldown, the enemy stops moving and tries to wind up an attack. If the player moves out of melee radius before the wind-up finishes, the attack is cancelled; otherwise, damage is applied, and the enemy attack goes on cooldown.

In the EnemyHealth script, the current health of an enemy is reduced whenever damage is taken, and death is triggered when it reaches zero. An OnDeath event is triggered when the enemy dies. The player also gains experience, and the kill counter increases. The enemy game object is then destroyed once the death animation plays.

4.5 Player Movement and Combat

Unity's Input System handles player controls: WASD for movement, the left mouse button for attacking, and the scroll wheel for camera zoom. The Cinemachine module handles camera behaviour and restrictions, such as limiting axis rotation and stopping the camera from clipping through the walls.

In the PlayerCombat script, when an attack input is performed, the script checks if the player is already attacking. If not, an attack animation is triggered, and the speed of the animation is adjusted based on the player's attack speed multiplier. The MeleeDamage() function uses Physics.OverlapSphere() to detect all enemies within the specified melee radius and apply damage based on the player's melee damage stat. This function isn't called immediately; it's called by an animation event near the end of the animation clip, so it syncs up correctly with the attack swing. SpawnAttackVFX() is also called through the animation and creates a hit visual effect.

4.6 Dynamic Difficulty Scaling

The difficulty scaling revolves around the GameTimer script. GameTimer continuously ticks down and updates the UI, and when it reaches zero, it raises the TimerFinished event.

Both difficulty-scaling scripts monitor the GameTimer and trigger difficulty spikes when the time falls below specified thresholds. These difficulty spikes make enemies stronger by increasing their max health and melee damage by finding all EnemyHealth and EnemyCombatMelee components and applying a percentage-based increase. By modifying all existing components, all enemies on the map become stronger when the threshold is reached.

Both scripts reduce respawn delay and increase elite spawn chance, but BSPDifficultyScaling increases the number of enemies in a room, while CADifficultyScaling increases the minimum number of enemies spawning near the player. This makes runs progressively harder as GameTimer ticks down.

4.7 Objectives and Game States (WIN/LOSE)

Collecting keys is an important objective for activating the portal on stage one. KeySpawner places a set number of keys in randomly selected BSP rooms, preventing multiple keys from appearing in the same room or in the spawn room. It also reduces the number of keys spawned if the specified amount to spawn is greater than the number of rooms. KeyController tracks the number of spawned and collected keys, while KeyPickup tracks when the player picks up the key.

PortalController is responsible for spawning an inactive portal in the starting room and continuously checking whether the required number of keys has been collected and the required kill count has been reached. If both these conditions are met, the inactive portal is replaced with an active version. The PortalTrigger script then handles the stage

transition by saving the player's current state in PlayerData and loading the next scene when the player enters the portal.



Figure 15 Inactive Portal



Figure 16 Activated Portal

On player death, the game-over panel is shown, and the game is paused using `Time.timeScale = 0`. The player can either retry their current stage or return to the main menu. If a player dies on stage 2, the saved checkpoint is reloaded before the stage restarts, so their stats aren't reset to the default values.

The win state is implemented in Stage 2 through `CATimer`, which listens for `GameTimer.TimerFinished` and initiates a `PlayerWon` event. `GameManagerUI` then responds to this event by showing a victory screen, changing the retry button to restart, and pausing the game.



Figure 17 Win screen when timer reaches 0 on stage 2

4.8 User Interface and Supporting Systems

The MiniMap script is one of the main supporting systems. Instead of using a live camera view, it generates a texture directly from the dungeon data. Each grid cell is represented as a pixel whose colour depends on the grid type. The script also converts the player and portal coordinates into UI positions on the map.

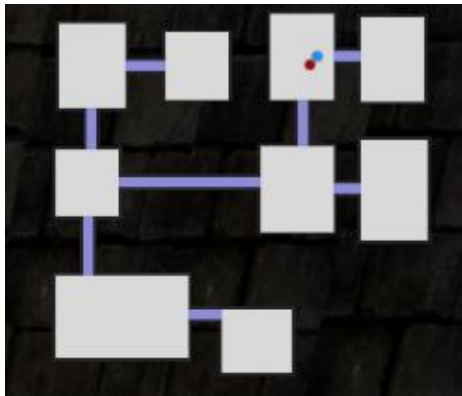


Figure 18 CA mini map

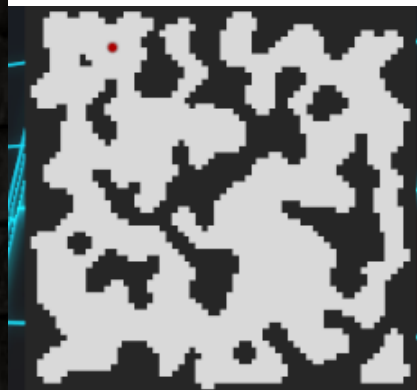


Figure 19 BSP mini map

PauseMenuUI is responsible for freezing the game when escape is pressed and for displaying statistics generated by LogMetrics. These metrics provide various data on the generation methods, such as the number of rooms and the percentage of the map that is a floor.

MainMenuUI includes a stage selection panel that allows players to load the second scene without advancing the game. Before loading a stage, it resets the run data by calling `PlayerData.Instance.ResetRunData()`, which ensures each run starts with default values.

MusicManager controls the background music of each stage. It checks each frame whether the player is in combat and smoothly fades between two audio sources accordingly.

5. Evaluation

Word Count: 2271/2500

5.1 Evaluation Aims

This chapter evaluates the completed prototype based on the project's original aim. It assesses how well the main systems and generation techniques contributed to creating a replayable roguelike experience. It examines whether both procedural generation methods produce distinct, playable levels and whether the core gameplay mechanics provide a balanced experience.

5.2 Evaluation Methodology

The prototype was evaluated using a combination of technical systems testing and user playtesting. This allowed for the discovery of any major bugs while also gathering feedback on how well the prototype performed in practice.

5.2.1 Technical Testing

Technical testing involved gameplay and system testing after every major iteration. This meant that whenever a major system was implemented, multiple game runs were performed to verify that the system worked as expected. Any bugs and issues encountered were noted, and a priority order was created based on what needed the most attention. Game-breaking and other major bugs had higher priority over minor bugs and polish. Every time an issue was resolved, the testing would be repeated to ensure it was truly fixed and didn't create further issues. These technical tests were not just limited to discovering bugs; systems that did not function well were also noted for future revisions. Both map generation techniques were extensively tested to ensure map connectivity. This testing involved repeatedly generating the map under different inspector settings. Repeated generations were necessary because of the procedural nature of the game, where issues in generation would not always manifest. Once the prototype was mostly complete, full play-throughs were conducted to ensure the game could be played from start to finish without any issues.

5.2.2 User Testing

User testing involved collecting feedback from a small number of individuals. This process followed an early-access approach to testing and development, a method increasingly common in modern games. Playtesters were notified that some aspects of the game were incomplete but were instructed to critique the systems currently available. This meant player feedback was gathered and used to shape the game's future, rather than just for bug fixing and testing. The result was a game direction that

was significantly different from what was originally planned.

An important consideration when selecting players for testing was choosing both family/friends and strangers. This ensured that the collected feedback was critical and reduced bias stemming from personal ties. As for the feedback itself, it was collected through player surveys at the end of the playthrough and through think-aloud testing, in which players were instructed to speak their thoughts aloud while playing. The survey allowed users to rate their experience with the core systems and to provide feedback or highlight any pain points. This meant feedback on specific elements of the game was collected as they were encountered, allowing for a real-time understanding of how players interacted with the game system and mechanics. Furthermore, instructions given to the playtesters were kept to a minimum to prevent influencing the feedback, therefore strengthening the validity of the user tests.

5.3 Evaluation of Procedural Map Generation

The project aimed to compare two procedural generation techniques in a roguelike game. These techniques were evaluated based on their ability to produce layouts that felt distinct from one another while remaining playable. There was a particular focus on how these different layouts influenced gameplay and ease of navigation.

5.3.1 BSP Evaluation

The BSP system performed well, creating a reliable, structured, and easy-to-navigate dungeon layout. There was a clear distinction between rooms and corridors, and the dungeon had no connectivity issues. This meant the player could traverse all areas of the generated dungeon without any blocked paths. Repeated testing with different inspector settings for dungeon generation consistently yielded stable outcomes. This highlighted the main strength of BSP: a good balance between variety and consistency. User surveys supported this conclusion, with most players noting that they never felt lost while exploring.

Despite these strengths, my BSP implementation had notable technical issues. Each grid cell was a 1 x 1 block, resulting in a very large number of wall and floor prefabs being instantiated. This proved extremely CPU-intensive, as each prefab included its own mesh collider, resulting in many physics calculations that noticeably slowed the game. I reduced the number of calculations by removing the mesh collider for each floor and using a single large mesh collider that covered the entire map; however, each wall column still retained its own mesh collider for simplicity, at the cost of performance. On the non-technical side, the uniformity and consistency resulted in monotonic structures where every room looked similar, just in different sizes. To minimise the

effects of this issue, I added an offset to the wall textures, resulting in slight variations in their appearance and making the rooms look more interesting and less monotonous.

5.3.2 CA Evaluation

CA produced a noticeably different style of level generation. In contrast to BSP, it created maps that felt more organic and chaotic. Players noted that this distinction made environments feel more interesting and visually appealing. This was reflected in gameplay, where players often had to adapt to the unpredictable map layout, creating a more varied experience, albeit more difficult. While I consider this a notable strength of CA and the desired outcome, it did come with a minor drawback. Sometimes players were placed in small, narrow areas, severely limiting their mobility and cutting them off from the rest of the map. I remedied this by increasing the smoothing value in the CA algorithm, resulting in larger playable areas. While it reduced the issue significantly, there was still a low chance that a player would have to fight in a small area. Overall, I found the CA implementation a success despite occasional issues; it met the design goal of creating a noticeably different experience.

5.3.3 Comparison of BSP and CA

Both map generation techniques proved successful, but in different ways. BSP was more stable and structured, while CA was the opposite, but offered a more interesting gameplay and visual experience. Where BSP was praised for its ease of navigation, CA felt short in that regard. On the other hand, CA offered a more replayable experience than BSP; each generation produced a map that was more distinct and less recognisable than the previous generations.

Survey results showed a divide in opinion on which generation method felt better; some preferred the more traditional dungeon-crawling experience, while others preferred the fresh, less predictable experience offered by CA. Due to subjective differences in art style and music, as well as the relatively small survey sample size, no generation method could be considered objectively better than the other.

7. Which map generation style did you prefer?

7 responses

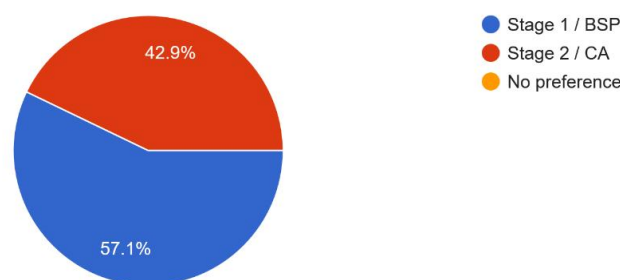


Figure 20 Player preference between BSP and CA map generation techniques.

5.4 Evaluation of Gameplay Systems

The gameplay systems played a significant role in shaping the player experience and highlighting the differences between the two map-generation techniques. Together, these systems determined how the player interacted with the generated environment, allowing for feedback on how each generation technique affected gameplay.

5.4.1 Combat, Enemies and Difficulty Scaling

Combat underwent the largest change from initial implementation to complete prototype. It was initially designed as an autobattler in which the player character would automatically attack enemies within range. Due to scope limitations, the number of attacks was limited, resulting in an unsatisfactory experience. As evidenced by the first round of playtesting, players reported a lack of agency because they could only run around the map. This created a choice between adding more attacks, potentially introducing new bugs, or switching to a player-controlled attack type while keeping the number of attacks limited. In the end, the latter was implemented, and in the second round of playtesting, it received a positive response.

For enemies, initially, there was just one type of enemy with the same appearance on both maps. These enemies remained constant and did not grow stronger over time, which made the game feel slightly repetitive and too easy. Based on observations of players, there was little incentive to experiment with builds, as players did not need to do so to overcome the game's challenges. Taking this feedback into account, I implemented an elite enemy type that was significantly stronger than regular enemies, along with difficulty scaling that made the game harder over time. As a result, players felt more incentivised to try out different builds, leading to the formation of various playstyles. To address visual repetition, I decided to change the regular enemy's appearance to match the map theme. Players responded positively to this change, as it introduced variance even if the enemies were functionally the same.

5.4.2 Progression, Stats and Upgrades

The progression and upgrade system was also affected by feedback on combat. Initially, it was being designed to allow players to choose from various attacks, typical of an autobattler roguelike. After the shift to a more hack-and-slash combat style, the upgrade system had to be changed to allow individual stat upgrades. This added more depth to combat as players responded positively to the addition of more playstyles in future playtests.

The attack speed stat was particularly noteworthy. In early technical testing, I noticed it was far stronger than the other stats. This was due to how the attack speed upgrade was initially calculated; its linear growth could scale disproportionately to the other stats if chosen repeatedly. This created a balancing issue since there was little incentive to choose any of the other weaker stats. To resolve this, I decided to introduce diminishing returns through the upgrade formula, meaning any subsequent upgrades would be slightly less effective, making it more comparable to the other stats.

5.5 Performance

As stated above, one of the biggest performance deterrents was the large number of mesh colliders and physics calculations. Adding on to this was the lighting and how it interacted with skinned meshes. It is common to have many entities on screen, and lighting bounce calculations for each one can become costly in performance. To reduce the effects of this and to increase the frame rate, I created lighting layers and then excluded enemies from certain layers. This meant there was less interaction between the light and enemies, resulting in noticeably better performance.

On the technical side, I optimised some functions to run more efficiently; for example, the `GetRandomFloorPosition()` in `CADungeonGenerator` was optimised to pick random spots and try to spawn the player there, rather than manually looping through every cell until it found a floor cell. This meant faster loading on scene change most of the time.

5.6 User Feedback and Results

A user survey was conducted once the prototype was in its final stages, with 7 participants playing through the game and providing feedback. Although the sample size was small, it included players with varying levels of familiarity with the roguelike genre, allowing evaluation of how well each procedural generation method performed in a roguelike game.

1. Have you played roguelike or roguelite games before?

7 responses

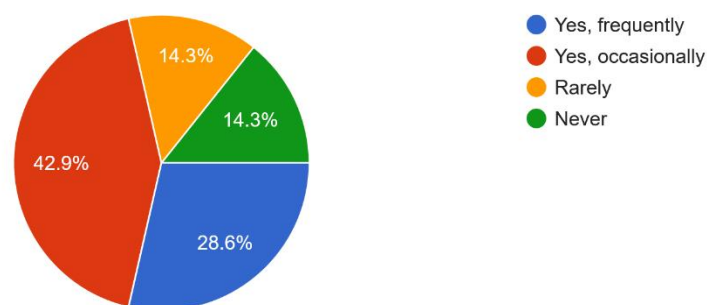


Figure 21 Playtester familiarity with roguelike games

One of the project's goals was to create two PCG maps that felt distinctly different. User surveys indicate this was largely a success.

4. Did the two stages feel noticeably different from each other?

7 responses

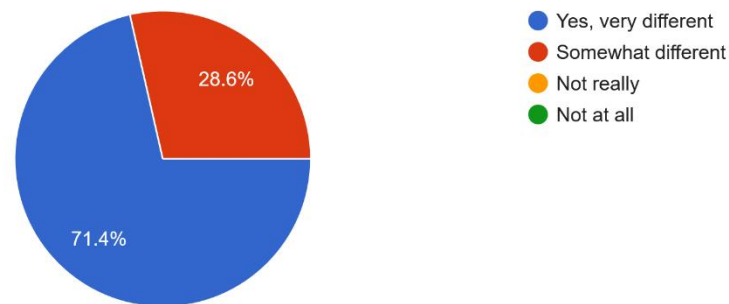


Figure 22 Player responses on whether the two stages felt noticeably different

User testing also concluded that the BSP map was easier to navigate, while the CA map was generally more visually interesting.

5. Which map did you feel was easier to navigate?

7 responses

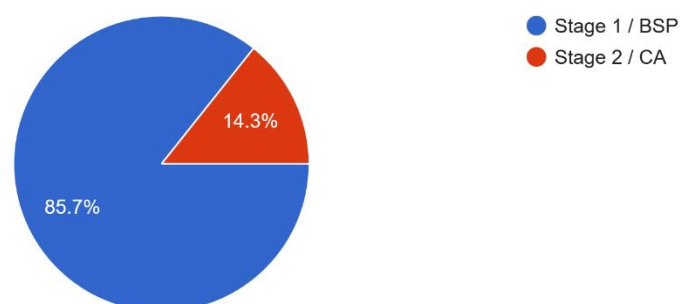


Figure 23 Survey results showing which map players found easier to navigate.

6 Which map was more visually interesting?

7 responses

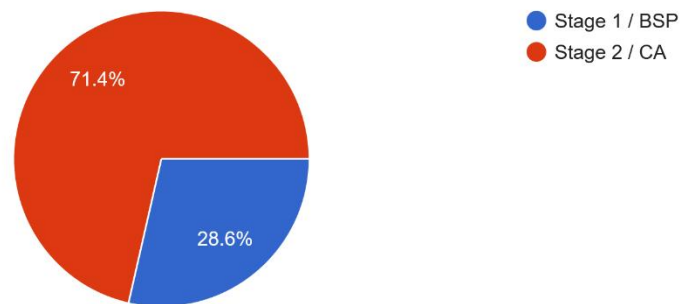


Figure 24 Survey results showing which map players found more visually interesting.

A comparison before and after the combat update shows a clear improvement in user satisfaction rating. This comparison was conducted by having the same 7 people test an earlier build of the prototype and answer questions about it.

5. How satisfying did combat feel?

7 responses

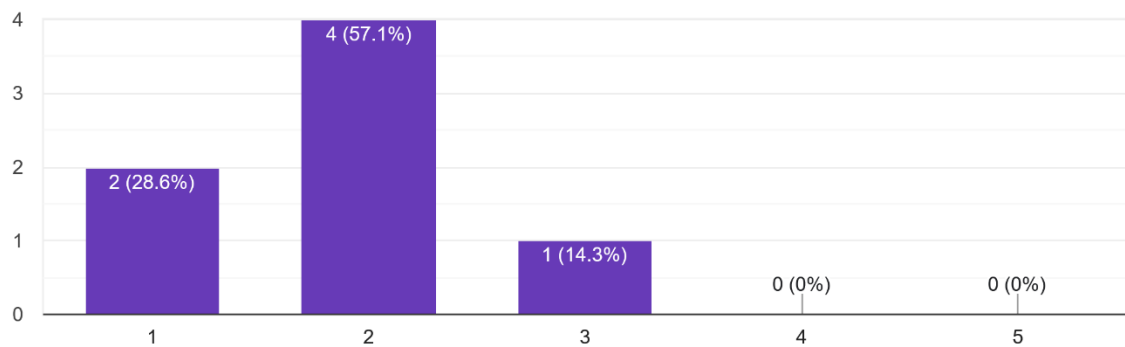


Figure 25 Graph showing combat satisfaction rating before the combat update

9. How satisfying did combat feel?

7 responses

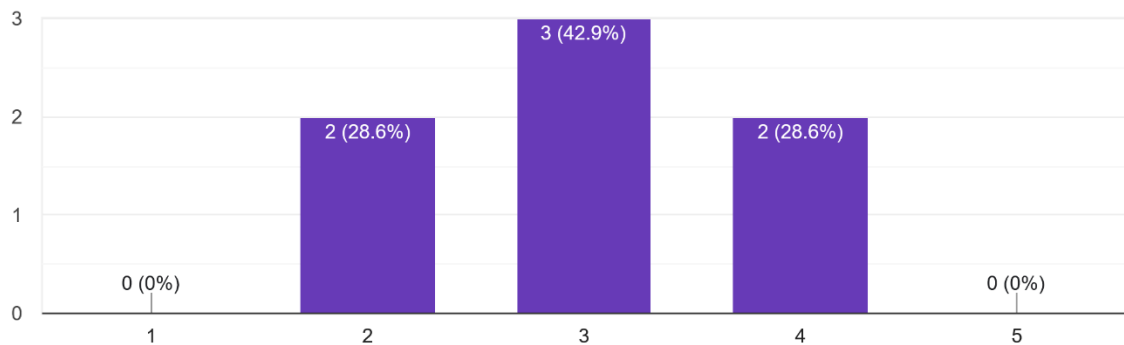


Figure 26 Graph showing combat satisfaction after the combat update.

Overall, the quantitative data collected through surveys enabled effective comparison of PCG map generation techniques. It provided insight into the project's shortcomings and the improvements made. The data also indicated that most players perceived the two PCG techniques used at each stage as significantly different, providing insight into user preferences between them.

5.7 Critical Discussion

Overall, the evaluation showed that the prototype met its main objective of creating a replayable roguelike experience. For BSP, these strengths were apparent, as it succeeded in creating a traditional dungeon-crawler experience, while CA introduced unpredictability, forcing players to adapt on each playthrough. The combat and difficulty scaling created a challenging experience that encouraged experimentation with builds to a satisfying degree. This is further supported by survey data, in which most players reported that the game was slightly hard and that they had to make different build choices.

11. How would you describe the overall difficulty of the game?

7 responses

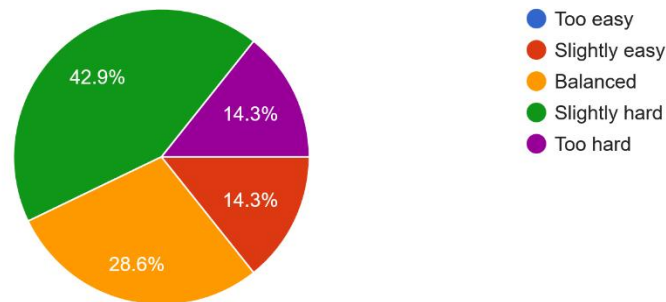


Figure 27 Graph showing how difficult players found the game to be

10. Did the upgrade system encourage different playstyles or build choices?

7 responses

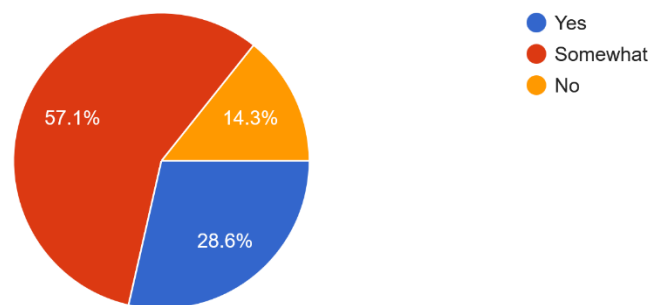


Figure 28 Graph showing how encouraged players felt about trying out different builds and playstyles

Along with these strengths were some limitations. The performance issue, while better, is still inefficient because it creates a collider for every single wall block. This is a flaw with the implementation of the PCG generation systems and fully resolving it would require a rewrite of the generation system. Another limitation discovered was due to how the project implemented procedural generation. By having all aspects of the environment generated procedurally and the dungeon built grid by grid, the dungeon often looked repetitive and empty, missing a hand-crafted feel.

5.8 Future Improvements

My project serves as the foundation for a full roguelike game. Many of the systems are designed to enable future development. For example, the stage-by-stage method used

to showcase PCG techniques can be expanded to include additional stages and techniques. This is further supported by the game's setting, which allows for a wide range of possibilities, with each stage set in a different thematic location.

Another improvement I could make is adding to the combat depth; the player stats system is designed for easy addition of more stat modifiers. Crit rate, which is the chance an attack has to do more damage, and crit dmg, which is the amount of extra damage an attack does when it crits, are popular stat modifiers found in many roguelikes that can contribute to variance and unpredictability in the game. Additionally, the player can be given more attacks, and more enemy types, such as ranged enemies, could be introduced, further allowing for more playstyles and varied encounters.

Finally, a reconstruction of the BSP implementation could allow for a more handcrafted feel by manually creating stylised rooms, then using BSP to randomly place them, resulting in a map that blends human-designed rooms with algorithmically generated structure. This hybrid approach to procedural generation could lead to better map design and address some of the limitations found in this project.

Conclusion

Word Count: 119/1000

This project set out to implement two PCG techniques and create a roguelike prototype. The project involved creating a working game that implemented both techniques and allowed a comparison of gameplay experiences. The final evaluation showed that these PCG map techniques not only increased variation across runs but also influenced how players interacted with the roguelike systems. The project also demonstrated the strengths and weaknesses of each and provided insight into their implementation in video games.

It was also educational, enabling the discovery of better processes for developing PCG, such as the hybrid approach combining hand-crafted levels with PCG. In conclusion, my project achieved the initial goals, but I was unable to objectively determine which PCG method was better.

BIBLIOGRAPHY

- [1] M. Hendrikx, S. Meijer, J. van der Velden, and A. Iosup. 2013. Procedural content generation for games: A survey. *ACM Transactions on Multimedia Computing, Communications, and Applications* 9, 1, Article 1 (February 2013). <https://doi.org/10.1145/2422956.2422957>
- [2] A. Gellel and P. Sweetser. 2020. A hybrid approach to procedural generation of roguelike video game levels. In *Proceedings of the Australasian Computer Science Week Multiconference (ACSW '20)*. <https://doi.org/10.1145/3373017.3402945>
- [3] H. Salinas. 2024. An exploration of procedural methods in game level design. Undergraduate Honors Thesis. Department of Computer Science and Computer Engineering, University of Arkansas, Fayetteville, AR, USA.
- [4] Internet Archive. 1978. Beneath Apple Manor (1978 version fixed). <https://archive.org/details/beneath-apple-manor-1978-version-fixed>
- [5] R. Francis. 2019. How much of a genius-level move was using binary space partitioning in Doom? *Ars Technica* (December 10, 2019). <https://arstechnica.com/gaming/2019/12/how-much-of-a-genius-level-move-was-using-binary-space-partitioning-in-doom/>
- [6] M. Abrash. n.d. Chapter 64: Quake's visible-surface determination. In *Michael Abrash's Graphics Programming Black Book*. <https://www.jagregory.com/abrash-black-book/#chapter-64-quakes-visible-surface-determination>
- [7] L. Caballero, R. J. Catchpole, J. A. Greenwood, E. D. Stout, and J. P. McFadden. 2016. Conway's "Game of Life" and the epigenetic principle. *Frontiers in Cellular and Infection Microbiology* 6, Article 57 (June 2016). <https://doi.org/10.3389/fcimb.2016.00057>

[8] B. M. F. Viana and S. R. dos Santos. 2021. Procedural dungeon generation: A survey. *Journal on Interactive Systems* 12, 1 (August 2021), 83–101. <https://journals-sol.sbc.org.br/index.php/jis/article/view/999>